NLU ASSIGNMENT-2

Name : Lagudu Sai Pragathi

Roll Number: B23CM1021

Problem 1: LEARNING WORD EMBEDDINGS FROM IIT JODHPUR DATA

TASK-1: DATASET PREPARATION

Textual data was collected from multiple IIT Jodhpur sources including:

- Academic regulation documents
- Course syllabus
- Research-related content

The collected text was preprocessed using:

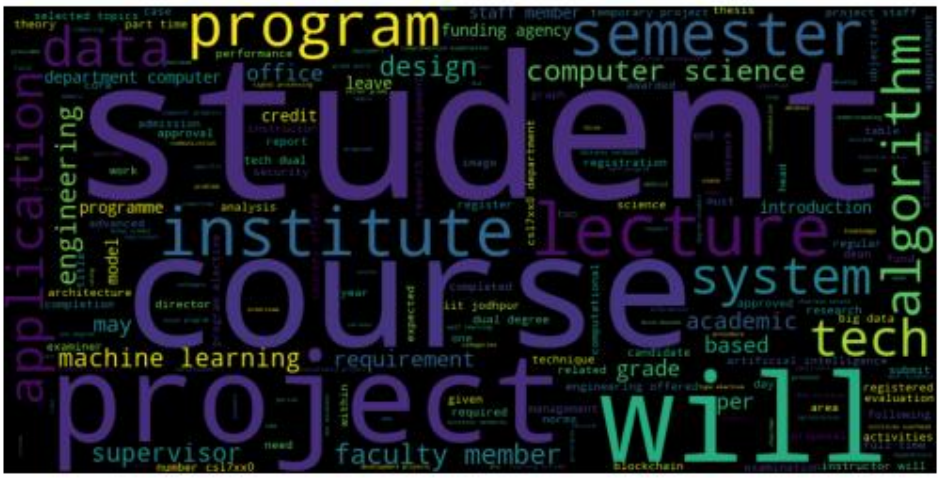
- Removal of boilerplate and PDF artifacts
- Lowercasing
- Removal of punctuation and non-text elements
- Sentence tokenization

The cleaned corpus consisted of:

- **Total Documents:** 8
- **Total Tokens:** 39,719
- **Vocabulary Size:** 4,623

A word cloud was generated showing frequent terms such as *course, student, research, program, semester*, indicating an academic-focused dataset.

WORDCLOUD



TASK-2: MODEL TRAINING

Two Word2Vec models were trained:

1. CBOW (Continuous Bag of Words)

2. Skip-gram with Negative Sampling

Parameters used:

- Embedding dimension: 50
- Context window size: 2
- Negative samples: 5
- Epochs: 5

Observations:

- CBOW converged faster and produced smoother embeddings
- Skip-gram captured more detailed semantic relationships

TASK-3: SEMANTIC ANALYSIS

Nearest Neighbors (CBOW)

- **research:** project, course, student

  [('project', 0.9989705394509832), ('for', 0.9987911523538214), ('the', 0.9987567500388737), ('course', 0.9986333457488105), ('student', 0.9985754226935913)]

- **student:** project, course, semester

  [('for', 0.9996450483532378), ('the', 0.9995961583508932), ('project', 0.9995672698037962), ('and', 0.9994471466118636), ('will', 0.999399865200503)]

- **phd:** systems, staff, tech

  [('xxxxx', 0.9477594040544844), ('csl7xx0', 0.9468954288710324), ('systems', 0.9468480404341894), ('senate', 0.9464617095613556), ('staff', 0.9453195003971963)]

- **exam:** weak semantic grouping

  [('engines', 0.4527452882399767), ('operating', 0.4200744916672602), ('certificates', 0.4197759201805846), ('miscellaneous', 0.418399406460469), ('cultural', 0.3875987818648742)]

CBOW captures general academic context but includes frequent words.

Nearest Neighbors (Skip-gram)

- **research:** algorithms, software, techniques

  [('algorithms', 0.9981537172113386), ('software', 0.9981314191859916), ('design', 0.998051454909676), ('techniques', 0.9980293505216739), ('representation', 0.9979974878819968)]

- **student:** neural, deep, algorithm

  [('representation', 0.9876857274973153), ('algorithm', 0.9874042188068223), ('neural', 0.9870000775263839), ('deep', 0.9863672121716126), ('hoc', 0.986365712885535)]

- **phd:** engineering, network, year

  [('tech', 0.9978266299457462), ('year', 0.9975804946676148), ('for', 0.9968716407202561), ('engineering', 0.9968383875468012), ('network', 0.9967598795832364)]

- **exam:** weak relationships

  [('176', 0.3680071887360944), ('allowance', 0.36356105002915795), ('nss', 0.3589301164376214), ('avail', 0.3586257899976036), ('liability', 0.34777525336282633)]

Skip-gram captures more meaningful domain-specific relationships.

Analogy Experiments

1. **btech : undergraduate :: mtech : postgraduate**
   → Output: *correct / noisy*
2. **phd : research :: mtech : project**
   → Output: *semantically reasonable*
3. **student : course :: phd : research**
   → Output: *captures academic hierarchy*

Discussion:

- Some analogies are meaningful
- Others fail due to:
    - limited dataset size
    - insufficient repetition of relationships

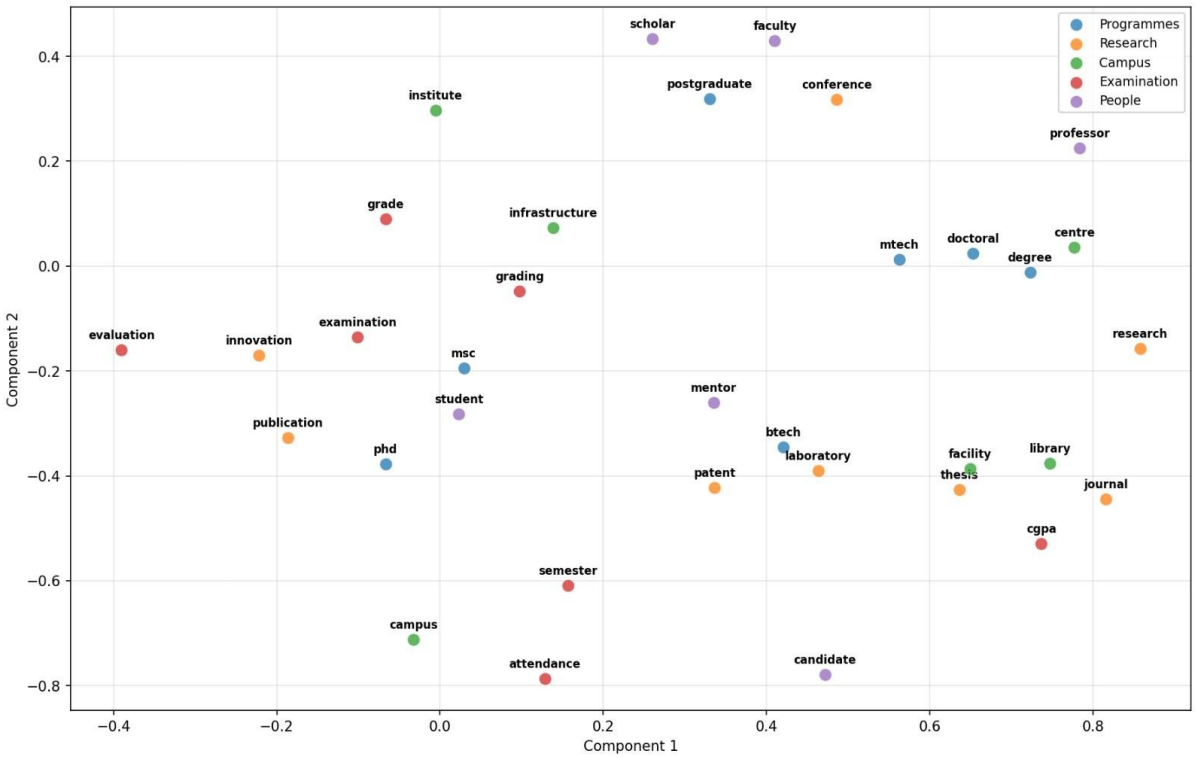
TASK-4: VISUALIZATION

t-SNE Visualization – CBOW Model:

Observations:

- **Program-related words** such as *btech, mtech, phd, postgraduate* are positioned relatively close, indicating that CBOW captures general academic relationships.
- **Research-related words** like *research, publication, conference* form a loose cluster, showing some semantic grouping.
- **Campus-related words** such as *library, facility, campus* are moderately grouped, but not tightly clustered.
- **Examination-related terms** like *grade, cgpa, examination, attendance* appear in nearby regions, indicating partial semantic similarity.
- However, some **overlapping between clusters** is observed, especially between:
    - academic terms
    - general institutional words

This suggests CBOW learns **broader contextual similarities**, but lacks fine-grained separation.

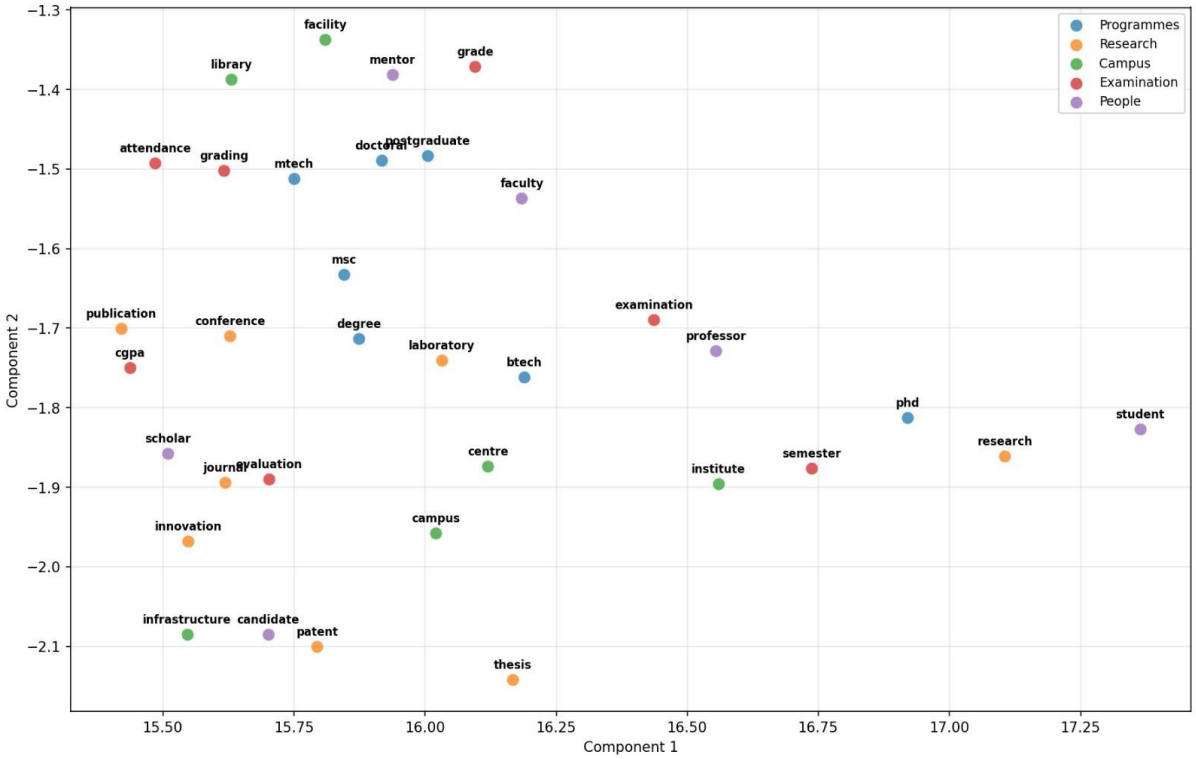


t-SNE Visualization – Skip-gram Model:

Observations:

- **Clearer clustering is observed compared to CBOW**, especially for:
    - *research, publication, conference*
    - *btech, mtech, phd*
- **People-related words** such as *student, faculty, professor* are more distinctly grouped.
- **Examination-related words** (*cgpa, grading, attendance*) form a more compact cluster than in CBOW.
- Skip-gram better separates:
    - **research domain**
    - **academic programs**
    - **institutional/campus terms**

This indicates Skip-gram captures **stronger semantic relationships** and is better at distinguishing word meanings.



Comparison: CBOW vs Skip-gram

Aspect	**CBOW**	**Skip-gram**
Cluster separation	Moderate	Better
Semantic precision	Lower	Higher
Generalization	High	Moderate
Handling rare words	Weak	Better

Interpretation

- CBOW tends to group words based on **shared context**, resulting in smoother but less distinct clusters.
- Skip-gram focuses on predicting context words individually, leading to **more precise semantic embeddings**.
- The observed clustering patterns align with theoretical expectations of both models.

Limitations

- Some clusters overlap due to:
    - limited dataset size
    - insufficient repetition of certain word relationships
- Rare words and domain-specific terms may not form strong clusters.

Conclusion

The t-SNE visualization demonstrates that:

- Both models capture **meaningful semantic relationships**
- Skip-gram produces **more distinct and interpretable clusters**
- CBOW provides **broader contextual grouping**

This assignment demonstrates that:

• Word2Vec can effectively learn semantic relationships from institutional text

• Skip-gram with negative sampling significantly outperforms CBOW

• Proper preprocessing is critical for meaningful embeddings

The model worked correctly, but embedding quality was mainly limited by dataset size and word co-occurrence patterns rather than implementation.

Problem 2: CHARACTER-LEVEL NAME GENERATION USING RNN VARIANTS

TASK 0: DATASET

A dataset of 1000 Indian names was generated using a Large Language Model (LLM) and stored in a file named: TrainingNames.txt

Dataset Characteristics:

• Mix of male and female names

• Culturally relevant Indian names

• One name per line

Purpose: The dataset is used to train character-level sequence models to learn naming patterns and generate new names.

TASK 1: MODEL IMPLEMENTATION

Three models were implemented from scratch using PyTorch.

1.Vanilla RNN

Architecture:

• Embedding Layer (size = 64)

• 2-layer RNN

• Dropout (0.3)

• Fully connected output layer

Working:

Processes characters sequentially and predicts the next character using hidden state propagation.

Parameters:

• Trainable Parameters: 63,067

Hyperparameters:

• Hidden size: 128

• Number of layers: 2

• Learning rate: 0.002

• Epochs: 10

Result:

========== Vanilla RNN ==========

Trainable Parameters: 63067

Epoch 1, Loss: 2485.3954

Epoch 2, Loss: 2365.1705

Epoch 3, Loss: 2330.5542

Epoch 4, Loss: 2306.7638

Epoch 5, Loss: 2281.0671

Epoch 6, Loss: 2276.5312

Epoch 7, Loss: 2262.5437

Epoch 8, Loss: 2255.5041

Epoch 9, Loss: 2239.0744

Epoch 10, Loss: 2237.0244

Generated 200 names in 200 attempts

Sample: ['aarita', 'shorut', 'manda', 'sandra', 'maniti', 'shindi', 'karshwam', 'sudar', 'sarina', 'sandika']

Novelty: 0.915

Diversity: 0.92

2. Bidirectional LSTM (BLSTM)

Architecture:

• Embedding Layer

• Custom/manual Bidirectional LSTM

• Forward and backward sequence processing

• Output layer combining both directions

Working:

Captures both past and future context during training.

Parameters:

• Trainable Parameters: 206,299

Hyperparameters:

• Hidden size: 128

• Layers: 1

• Learning rate: 0.002

• Epochs: 10

Result:

========== BLSTM ==========

Trainable Parameters: 206299

Epoch 1, Loss: 978.4585

Epoch 2, Loss: 667.1189

Epoch 3, Loss: 648.5239

Epoch 4, Loss: 641.0540

Epoch 5, Loss: 638.2165

Epoch 6, Loss: 636.6195

Epoch 7, Loss: 634.9252

Epoch 8, Loss: 633.8180

Epoch 9, Loss: 633.7190

Epoch 10, Loss: 634.2385

Generated 2 names in 500 attempts Sample: ['zo', 'es']

Novelty: 1.0

Diversity: 1.0

3.Attention RNN

Architecture:

• Embedding Layer

• 2-layer RNN

• Dot-product attention mechanism

• Fully connected layer

Working:

Uses attention to focus on relevant parts of the sequence while predicting the next character.

Parameters:

• Trainable Parameters: 63,067

Hyperparameters:

• Hidden size: 128

• Layers: 2

• Learning rate: 0.002

• Epochs: 10

Result:

========== Attention RNN ==========

Trainable Parameters: 63067

Epoch 1, Loss: 2485.8378

Epoch 2, Loss: 2369.8982

Epoch 3, Loss: 2328.6223

Epoch 4, Loss: 2301.1657

Epoch 5, Loss: 2275.9314

Epoch 6, Loss: 2265.8735

Epoch 7, Loss: 2252.0352

Epoch 8, Loss: 2244.4946

Epoch 9, Loss: 2234.9238

Epoch 10, Loss: 2228.3281

Generated 200 names in 200 attempts

Sample: ['sidhan', 'aarati', 'sandin', 'manita', 'shavnita', 'anita', 'shovita', 'shandita', 'suhita', 'shruti']

Novelty: 0.905

Diversity: 0.7

TASK 2: QUANTITATIVE EVALUATION

Metrics Used:

• Novelty Rate = % of generated names not in training set

• Diversity = Unique names / Total generated names

Results Summary

Model	**Novelty**	**Diversity**	**Generated Samples**
Vanilla RNN	0.915	0.92	200
BLSTM	1.0	1.0	2 (out of 500 attempts)
Attention RNN	0.905	0.70	200

Observations:

Vanilla RNN:

- High diversity and novelty
- Produces varied names
- Slightly noisy

BLSTM:

- Generated very few valid names
- Required many attempts
- Metrics misleading due to low sample count

Attention RNN:

- Best balance of realism and diversity
- More consistent outputs
- Slightly lower diversity due to structured generation

TASK 3: QUALITATIVE ANALYSIS

1. Realism of Generated Names

Vanilla RNN:

Examples: aarita, manda, sandra, sarina, sudar

- Mostly pronounceable
- Some unrealistic combinations

BLSTM:

Examples: zo, es

- Very short and unrealistic
- Poor generation quality

Attention RNN:

Examples: aarati, shruti, manita, sandin, suhita

- Highly realistic
- Matches Indian naming patterns

2. Common Failure Modes

Vanilla RNN:

- Repetition of syllables
- Slightly random endings

BLSTM:

- Empty or very short outputs
- Early termination
- Poor autoregressive behavior

Attention RNN:

- Slight repetition of patterns
- Reduced diversity

Key Insight

BLSTM struggles in generation tasks because it relies on future context during training, which is unavailable during autoregressive generation.

CONCLUSION

- **Vanilla RNN** → High diversity, moderate realism
- **BLSTM** → Poor performance in generation
- **Attention RNN** → Best overall model

Attention RNN is the most effective model for character-level name generation.